Android Vision App Using Classical Image Processing

OpenCV and Smartphone Sensors (No Pretrained Models)

AIoT Lab
Instructor: Dr. Vasanth Iyer

February 6, 2026

Abstract

This project develops an Android application that performs real-time image processing using only classical computer vision techniques implemented with OpenCV and built-in smartphone sensors. The application avoids pretrained machine learning models and cloud-based AI services. Camera input is combined with inertial sensors to implement edge detection, motion tracking, and stabilization in a privacy-preserving and transparent manner.

1 Introduction

Many modern mobile vision applications rely on pretrained deep learning models. While effective, these approaches often obscure algorithmic foundations and raise privacy concerns. This project emphasizes classical image processing methods that operate directly on pixel data and combine sensor measurements to enhance robustness. The result is a fully on-device system that is efficient, explainable, and suitable for security-sensitive applications.

2 Project Objectives

2.1 Core Objectives

- Capture live camera frames on Android.
- Apply classical image processing using OpenCV.
- Integrate smartphone sensors (accelerometer, gyroscope).
- Visualize results in real time.
- Preserve user privacy through local computation.

2.2 Constraints

- No pretrained AI or deep learning models.
- No cloud-based processing.
- All computation must run on-device.

3 System Overview

The system consists of a camera acquisition module, an OpenCV-based image processing pipeline, and a sensor fusion module. Camera frames are processed sequentially while sensor data is sampled in parallel to stabilize and augment visual output.

4 System Flow Diagram (Student Guide)

Figure 1 illustrates the full processing sequence used in the application.

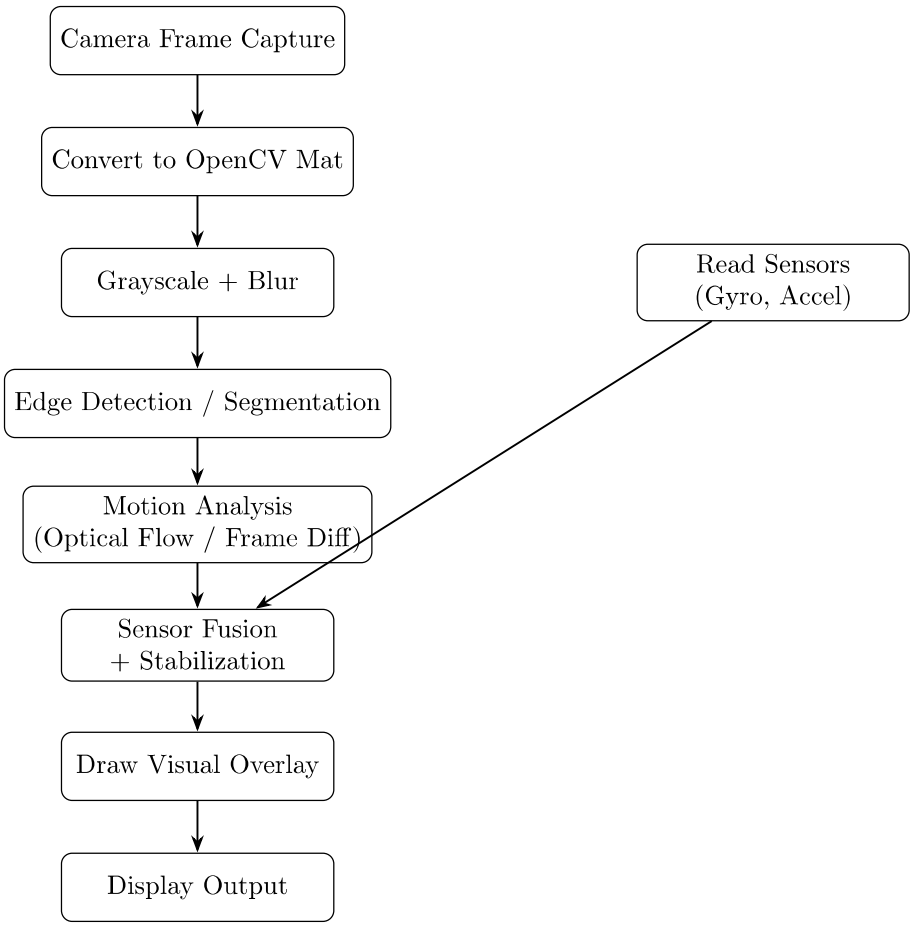

Figure 1: Classical OpenCV image processing pipeline with smartphone sensor integration.

Student Task: Map each block in Figure 1 to the corresponding OpenCV function or Android API call.

5 Classical Image Processing Pipeline

5.1 Preprocessing

$$I_g = \texttt{GaussianBlur}(\texttt{Gray}(I))$$

5.2 Feature Extraction

- Edge detection (Canny, Sobel)
- Thresholding and segmentation
- Contour extraction

5.3 Motion Analysis

Temporal changes are computed using frame differencing or Lucas–Kanade optical flow.

6 Sensor Integration

Sensor data is used to stabilize and augment vision output.

$$\hat{s}_t = \alpha s_t + (1 - \alpha)\hat{s}_{t-1}$$

6.1 Sensors Used

- Accelerometer
- Gyroscope
- Magnetometer (optional)

7 Language-Agnostic Pseudocode

```
Algorithm VisionWithSensors(frame, sensors):
    gray     Grayscale(frame)
    smooth     GaussianBlur(gray)

    edges     Canny(smooth)
    contours     FindContours(edges)

    motion     FrameDifference(previousFrame, frame)
    orientation     Smooth(sensors.gyroscope)

    overlay     DrawOverlay(edges, motion, orientation)

    previousFrame     frame
    return overlay
```

8 Kotlin (Android) Snippets

8.1 OpenCV Edge Detection

```
Imgproc.cvtColor(frame, gray, Imgproc.COLOR_RGBA2GRAY)
Imgproc.GaussianBlur(gray, gray, Size(5.0,5.0), 0.0)
Imgproc.Canny(gray, edges, 100.0, 200.0)
```

8.2 Sensor Listener

```
override fun onSensorChanged(event: SensorEvent) {
    if (event.sensor.type == Sensor.TYPE_GYROSCOPE) {
        gyroX = event.values[0]
        gyroY = event.values[1]
    }
}
```

9 Dart (Flutter) Snippets

```
img.Image gray = img.grayscale(inputImage);
img.Image blurred = img.gaussianBlur(gray, 5);
```

10 Evaluation

- Frames per second (FPS)
- Processing latency (ms)
- Stability under motion
- Battery usage

11 Privacy and Security Considerations

- No biometric identification is performed.
- No images are stored or transmitted.
- Processing is local and ephemeral.

12 Discussion

This project demonstrates that many real-time vision tasks can be implemented using classical algorithms without machine learning. Such systems offer transparency, lower computational cost, and reduced privacy risk.

13 Conclusion

The application presents a complete, on-device Android vision pipeline using OpenCV and smartphone sensors. By avoiding pretrained models, the project emphasizes algorithmic clarity, reproducibility, and privacy-aware design.

References

[1] OpenCV Documentation, https://opencv.org

[2] Android Sensor Framework, https://developer.android.com/guide/topics/sensors

[3] Szeliski, R., *Computer Vision: Algorithms and Applications.*